МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное Государственное бюджетное образовательное учреждение высшего профессионального образования
«Тихоокеанский государственный университет»

Кафедра «Математических методов защиты информации»

Анализ вредоносного программного обеспечения, защищенного методами упаковки и обфускации кода

Текстовый документ курсовой работы

по дисциплине

«Теория кодирования, сжатия и восстановления информации»

КР.220016171.ТД

Выполнил студент Малинский А.Б.

Факультет, группа ПОЛИТЕХ, КБ(с)-21

Руководитель работы ст. преп. Зуфарова А.С.

Виза: _____(доработать, к защите и т.д.)_____

_____

Хабаровск – 2026г.

КР.220016171.ТД

ЗАДАНИЕ

Изучить теорию, которая описывает процессы упаковки и распаковки кода программного обеспечения исполняемых файлов, форматы исполняемых файлов. Изучить основные методы, цели, подходы к применению упаковщиков, используемых в вредоносном программном обеспечении, а также последствия их использования. Освоить и показать на практике подходы к детектированию и работе с вредоносными файлами, в которых содержится упаковка.

КР.220016171.ТД

РЕФЕРАТ

Курсовая работа содержит 38 страниц текстового документа формата А4, включающего 5 рисунков, 8 таблиц, 13 использованных источников.

ВРЕДОНОСНОЕ ПО, АНТИ-ОТЛАДКА, УПАКОВКА, PORTABLE EXECUTABLE, ОТЛАДКА, СТАТИЧЕСКИЙ АНАЛИЗ, ДИНАМИЧЕСКИЙ АНАЛИЗ, ЭНТРОПИЯ, АССЕМБЛЕР, КОМПИЛЯЦИЯ, РЕВЕРС-ИНЖЕНЕРИЯ.

Целью работы является рассмотрение, изучение и работа с вредоносным ПО, которое защищено от проведения анализа методами упаковки кода исполняемого файла.

Способом исследования является изучение теории о структуре исполняемых файлов, принципах работы упаковщиков и подходов к изучению подобных файлов, методах их распаковки для дальнейшего анализа. Поиск специального инструментария, практическое применение методов на реальных образцах ВПО.

Работа выполняется с помощью самостоятельного изучения источников, посвященных реверс-инженерии и анализу вредоносного ПО, технической документации вендора целевой операционной системы.

Итоговый текст, представляющий собой обзор информации о предмете исследования, целях использования и процессах противодействия, может быть расширен как до исследовательской работы в области информационной безопасности, так и до учебно-методического пособия по реверс-инженерии и анализу ВПО.

КР.220016171.ТД

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 5
1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ АНАЛИЗА УПАКОВАННЫХ PE-ФАЙЛОВ 6
1.1 Структура исполняемого файла в ОС Windows 6
1.2 Принципы, методы и цели упаковки исполняемых файлов 10
2 ПРАКТИЧЕСКАЯ МЕТОДОЛОГИЯ ВЫЯВЛЕНИЯ И АНАЛИЗА УПАКОВАННОГО ПО 17
2.1 Признаки упаковки файла 17
2.2 Статический анализ упакованного ПО 17
2.3 Автоматизированное детектирование упаковки 20
2.4 О распаковке и восстановлении оригинальной программы 23
2.5 Tail Jump Catching 24
2.6 VirtualProtect Following 28
ЗАКЛЮЧЕНИЕ 31
Список использованных источников 33

ВВЕДЕНИЕ

Современный ландшафт киберугроз постоянно растёт. Злоумышленники, как и специалисты по информационной безопасности, используют совершенствующиеся подходы. Большой областью современного ландшафта является распространение вредоносного программного обеспечение. Совершенствуя подходы к разработке, их создатели пытаются скрыть ВПО от систем детектирования и усложнить процесс их анализа. Один из подходов, используемых для достижения этой цели – сокрытие кода исполняемых файлов.

Цели и задачи:

Цель: изучение теоретической информации и применение её на практике.

Задачи:

а) изучение структуры исполняемого файла;

б) изучение принципов упаковки и распаковки исполняемых файлов;

в) изучение влияния упаковки на анализ исполняемого файла;

г) изучение методов распаковки и применение их на практике.

Объект изучения: вредоносное программное обеспечение, разработанное для ОС Windows с применением упаковки кода исполняемого файла

В ходе изучения основными методами стали: изучение информации в источниках – практико-ориентированные пособия, техническая документация вендора, практическая работа на изолированном лабораторном стенде для изучения и анализа вредоносного ПО.

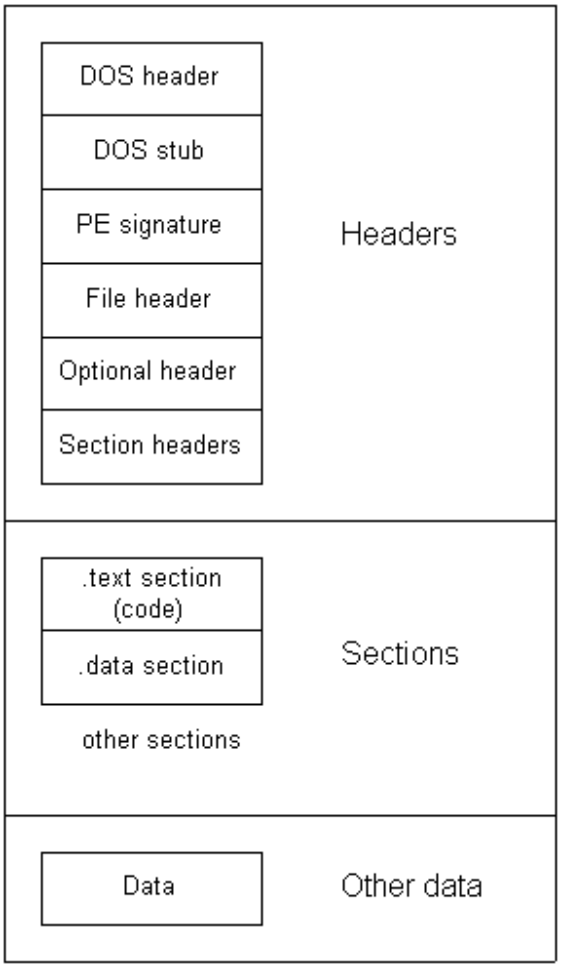
1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ АНАЛИЗА УПАКОВАННЫХ PE-ФАЙЛОВ

1.1 Структура исполняемого файла в ОС Windows

Формат Portable Executable (PE) - стандартный формат представления исполняемых файлов в операционных системах семейства Windows. Он применяется для хранения EXE-файлов, динамически подключаемых библиотек (DLL), системных драйверов и ряда вспомогательных программных модулей. Формат определяет способ размещения кода, данных, таблиц импорта и экспорта, механизм загрузки файла в адресное пространство процесса.

В контексте анализа вредоносного ПО и упакованных файлов формат PE представляет особый интерес, поскольку большинство методов сокрытия, сжатия и шифрования данных реализуется путем модификации стандартных структур формата. Без понимания внутреннего устройства PE-файла невозможно корректно как анализировать наличие и последствия упаковки, так и восстанавливать исходное состояние программы.

На рисунке 1 показана принципиальная структура исполняемого файла формата PE:



Структура IMAGE_DOS_HEADER располагается в самом начале файла и является наследием формата исполняемых файлов MS-DOS. Она начинается с двухбайтовой сигнатуры MZ 0x4d5a, по которой операционная система идентифицирует файл как исполняемый.

Наиболее значимым полем структуры является e_lfanew, которое содержит смещение до структуры IMAGE_NT_HEADERS. Это поле используется загрузчиком Windows и инструментами статического анализа для перехода к основной части PE-заголовков. Остальные поля структуры практически не используются по назначению, но физически присутствуют в файле.

С точки зрения анализа вредоносного ПО, IMAGE_DOS_HEADER может быть использован для:

- хранения вспомогательных данных;
- внедрения мусорной информации;
- маскировки сигнатур путем изменения неиспользуемых полей
- усложнения автоматизированного анализа.

Кроме того, некоторые вредоносные образцы намеренно повреждают или искажают структуру DOS-заголовка, рассчитывая на некорректную работу простейших парсеров PE-файлов

MS-DOS Stub Program представляет собой небольшой участок исполняемого кода, предназначенный для запуска в среде MS-DOS. В большинстве легитимных программ данный код выводит сообщение о невозможности запуска приложения в DOS-режиме.

В современных ОС эта часть не выполняет функциональной роли, но сохраняется в структуре файла для обратной совместимости. Это делает MS-DOS Stub привлекательной областью злоумышленников, в которой они могут:

- размещать зашифрованные данные;
- хранить первую стадию распаковщика
- внедрять механизмы анти-анализа
- маскировать пейлоад.

Поэтому наличие нестандартного или чрезмерного большого DOS-stub часто является косвенным признаком упаковки или обфускации.

Структура IMAGE_NT_HEADERS является центральным элементом PE-файла. Она начинается с сигнатуры PE\0\0, за которой следуют два ключевых заголовка: IMAGE_FILE_HEADER и IMAGE_OPTIONAL_HEADER. Данная структура определяет, каким образом файл будет отображён в память и выполнен процессором.

Сигнатура PE\0\0 однозначно указывает на начало основной PE-структуры. Её корректность критически важна для стандартного механизма загрузки Windows. Однако вредоносные программы для затруднения статического анализа иногда временно модифицируют или скрывают сигнатуру, восстанавливая её уже в памяти, что затрудняем статический анализ.

IMAGE_FILE_HEADER одержит общую информацию о файле, включая архитектуру целевого процессора, количество секций, временную метку компиляции и набор флагов, описывающих характеристики файла. При упаковке файла количество секций может быть искусственно уменьшено, а временная метка подменена или обнулена. Это затрудняет определение происхождения файла и его корреляцию с известными образцами.

IMAGE_OPTIONAL_HEADER содержит параметры, определяющие загрузку PE-файла в память. Данный заголовок является обязательным для исполняемых файлов. Его ключевые поля:

- AddressOfEntryPoint – адрес точки входа;
- ImageBase – базовый адрес загрузки
- SectionAlignment, FileAlignment
- Таблицы импорта, экспорта и ресурсов

Практически всегда упаковщики изменяют структуру AddressOfEntryPoint, перенаправляя выполнение на код распаковщика. После восстановления оригинального кода управление передается на исходную точку входа, которая называется Original Entry Point (OEP).

Таблица секций IMAGE_SECTION_HEADER описывает логическое разбиение PE-файла на отдельные области, каждая из которых выполняет определенную функцию. Типичные секции: .text, .data, .rdata, .rsrc

Каждая запись структуры IMAGE_SECTION_HEADER содержит:

- имя секции;
- виртуальный и физический размеры;
- смещение в файле;
- флаги доступа.

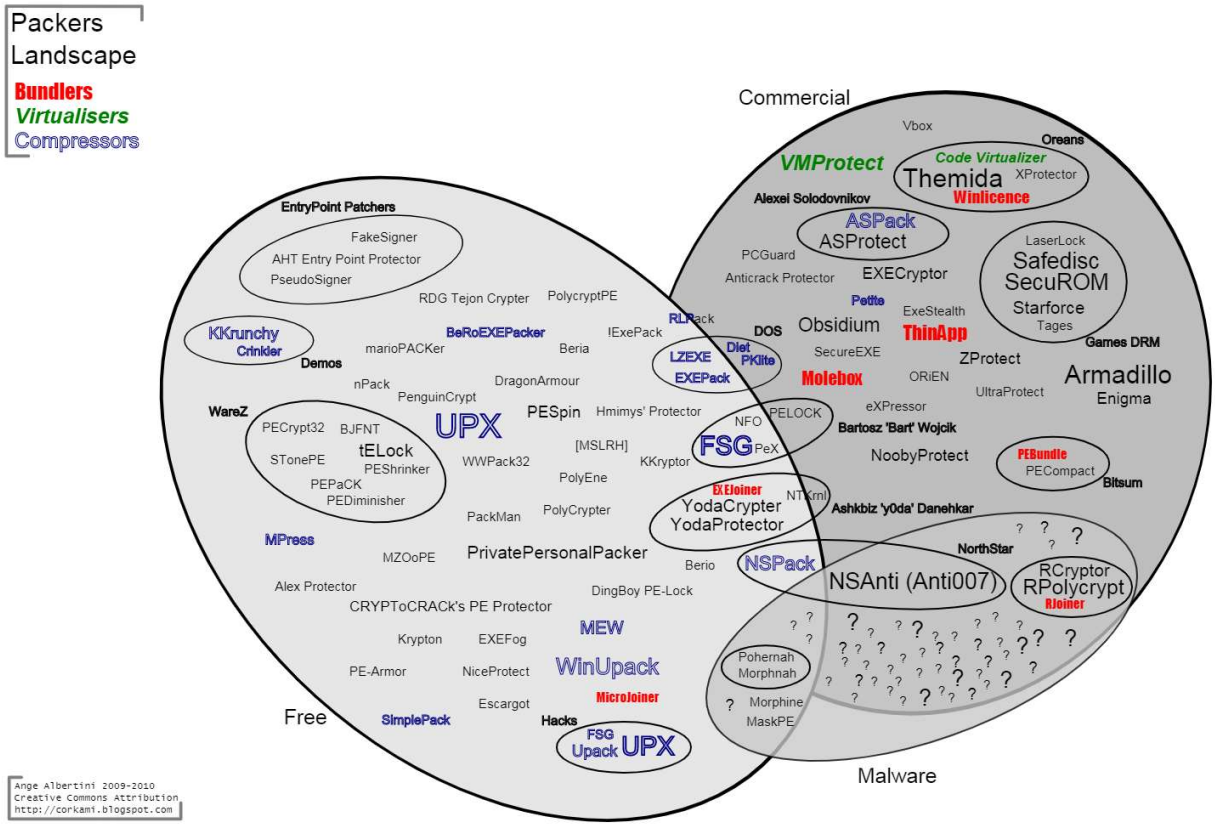
При упаковке структура секций изменяется наиболее заметно, что проявляется в:

- нестандартных именах секций;
- объединении кода и данных в одну секцию
- установка флагов исполняемости и записи одновременно (RWX);
- высокая энтропия секции;

1.2 Принципы, методы и цели упаковки исполняемых файлов

1.2.1 Определение и классификация упаковщиков

Упаковщики - инструменты или алгоритмы, предназначения для выполнения упакованного процесса. В настоящее время упаковщики в основном используется для защиты исходного кода программ. Они используются как производителями ПО, так и злоумышленниками. Существуют как коммерческие упаковщики, так и пользовательские, которые написаны самими злоумышленниками. Некоторые из них имеют открытый исходный код. Ландшафт упаковщиков представлен на рисунке 2:



Их классификация довольно сложная, так как один упаковщик может использоваться для нескольких целей. Однако все равно можно выделить наиболее узнаваемые группы упаковщиков по следующим классам:

1) Сжимающие упаковщики (Compressed Packers) - упаковщики, уменьшающие размер файла. В прошлом такие упаковщики использовались для устранения ограничений, связанных с низкой скоростью Интернета и ограниченным объемом памяти устройств. Сейчас технология несколько устарела, так как размер исполняемых файлов не является серьезной проблемой

2) Шифрующие упаковщики, крипторы (Encrypted Packer, Cryptors) предназначены для шифрования исполняемого файла, что значительно усложняет извлечение информации с использованием методов статического

анализа вредоносных программ. Также средства шифрования создают трудности для систем безопасности при обнаружении малвари. Разработчики ВПО используют подобные упаковщики для того, чтобы сделать свое ПО полностью необнаруживаемым

3) Защитные упаковщики, протекторы (Protecting Packers, Protectors) - тип упаковщиков, используемых разработчиками легитимного (в основном) ПО. Таким образов вендор препятствует изучению исходного кода своих программ их модификации и взлому техниками реверс-инженерии. Такие упаковщики используют как алгоритмы сжатия, так и алгоритмы криптографической защиты информации.

1.2.2 Анатомия упаковщика

Все упаковщики принимают на вход исполняемый файл и на выходе отдают также исполняемый файл. Упакованный файл может быть сжат, зашифрован, или как-нибудь ещё изменён, усложняя процесс анализа с помощью методов реверс-инженерии. Большинство упаковщиков используют алгоритмы сжатия для уменьшения размера, как было указанно выше. Упаковщики, разработанные для усложнения анализа файла кроме его шифрования также могут использовать техники анти-реверс-инженерии, такие как анти-дизассемблирование, анти-отладка, анти-ВМ. Упакована может быть как весь исполняемый файл, вместе со всеми данными и секцией ресурсов, так и только секции кода и данных.

Чтобы сохранять функциональность исходной программы, упаковщики должен сохранять информацию об импортах. Она может храниться в любом формате - это зависит от реализации упаковщика.

1.2.2.1 Unpacking Stub

Неупакованные исполняемые файлы загружаются операционной системой. Когда же запускается упакованная программа, то в ОС загружается

распаковочная заглушка - Unpacking Stub, которая, в свою очередь, загружает оригинальную программу. Точка входа указывает не на оригинальный код программы, а именно на заглушку. Сам исходный код может храниться в одном или нескольких разделах - зависит от упаковщика.

Заглушка часто маленького размера, так как она не влияет на основной функционал программы, а её функция проста: распаковать упакованный файл. При попытке статического анализа упакованной инструмент будет анализировать её, а не исходную программу.

В целом, распаковочная заглушка выполняет три действия:

1) Распаковывает оригинальную программу в память

2) Выполняет все импорты исходного исполняемого файла

3) Передает управление на OEP (Original Entry Point)

1.2.2.2 Загрузка исполняемого файла

Когда обычный исполняемый файл загружается в память, загрузчик читает PE-заголовок на диске и для каждого раздела на основе этого заголовка выделяет память. Затем разделы копируются в выделенные области памяти. Упаковщики и распаковщики упакованных исполняемых файлов или форматируют PE-заголовок таким образом, чтобы загрузчик выделял место для разделов, которые могли бы быть получены от оригинальной программы, либо самостоятельно создают эти секции. Заглушка-распаковщик распаковывает код каждой секции и копирует его в выделенную память. Конкретный метод зависит от целей упаковщика, и обычно содержится в заглушке.

1.2.2.3 Обработка импортов

Неупакованный PE-файл включает в себя секцию, которая сообщает загрузчику, какие функции импортировать, и секцию, в которой хранятся адреса имен всех импортируемых функций. Загрузчик Windows считывает

эту информацию и определяет, какие функции необходимы, и вставляет их по заранее определённым адресам.

Но загрузчик не может прочитать информацию, которая упакована. В упакованных программах информацию об импортах обрабатывает заглушка.

Конкретный подход зависит от упаковщика, но наиболее распространенный - импортирование функций LoadLibrary и GetProcAddress. После того, как заглушка распакует оригинальную программу, она прочитать оригинальную информацию об импорте. Вызывая LoadLibrary для каждой библиотеки, чтобы загрузить DLL в память, и после использует вызовы GetProcAddress для получения адресов каждой функции.

Другой распространенный подход заключается в том, чтобы сохранить исходную таблицу импорта, чтобы загрузчик мог читать информацию без распаковки заглушки. Но он не имеет никакой скрытности - при статическом анализе выявляется весь исходный импорт. Кроме того, так как импортируемые функции хранятся в исполняемом файле в виде открытого текста, то сжатие не будет оптимальным

Третий подход - сохранение одной функции импорта из каждой DLL, содержащейся в исходной. Он более незаметен, так как выявляется только одна функция для каждой библиотеки, но сами библиотеки определяются в полном объеме. Он проще первого для реализации, так как DLL не нужно загружать с помощью заглушки, но все равно обработка основного объема импортов будет происходить в ней.

Последний подход - удаление вообще всего импорта. В таком случае упаковщик должен найти все необходимые функции из библиотек без использования их имен, или, например, найти LoadLibrary, GetProcAddress и использовать их для дальнейшей обработки. Фактически, это полностью описывает работу шеллкода. Упакованная с таким подходом программа

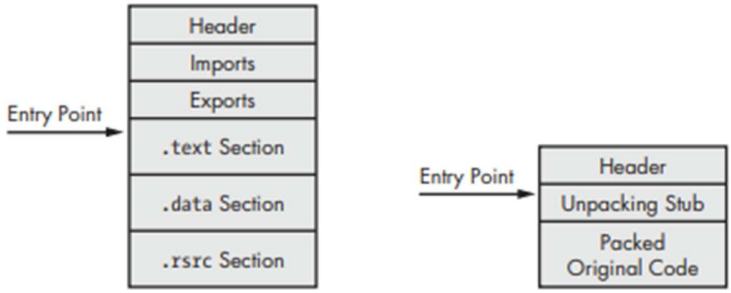
вообще не включает импорт - это делает её максимально незаметной. Но для этого распаковочная заглушка будет очень сложной в реализации и имплементации.

1.2.2.4 Tail Jump

Когда распаковочная заглушка завершит процесс распаковки, она должна передать управление оригинальной точке входа (OEP, Original Entry Point). Паттерн такой передачи управления на OEP известен как tail jump. Ассемблерная инструкция jump - наиболее простая и популярная для передачи управления. Из-за этого многие нелегитимные упаковщики скрывают её с помощью инструкций call и ret. Иногда tail jump маскируется с помощью функций ОС для передачи управления, например NtContinue или ZwContinue

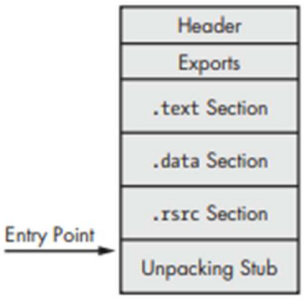
1.2.2.5 Иллюстрация процессов упаковки и распаковки

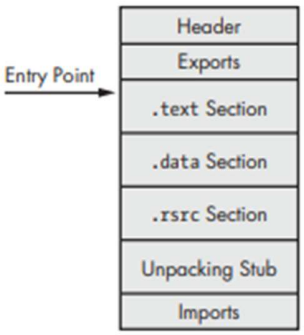
Слева на рисунке 3 - оригинальная программа. Заголовки и секции видны, точка старта программы установлена на OEP. Справа - упакованная программа в том виде, в котором она существует на диске. Всё, что о ней известно - новый заголовок, заглушка и упакованный исходный код.



Слева на рисунке 4 - программа после распаковки и загрузки в память. Заглушка распаковывает все необходимое для запуска кода. Точка старта программы указывает на заглушки, импорты отсутствуют. Справа -

полностью распакованная программа. Таблица импортов реконструирована, а точка старта снова указывает на OEP.





2 ПРАКТИЧЕСКАЯ МЕТОДОЛОГИЯ ВЫЯВЛЕНИЯ И АНАЛИЗА УПАКОВАННОГО ПО

2.1 Признаки упаковки файла

Индикаторы упаковки:

- Программа имеет мало импортов, в частности, если в импорте только LoadLibrary и GetProcAddress
- Когда программу открывают в дизассемблере или декомпиляторе, только малая часть кода распознана с помощью автоматического анализа
- Когда программа запускается через отладчик, то появляется предупреждение о том, что программа может быть упакована
- Программа имеет имена секций, который относятся к известным упаковщикам
- Программа имеет ненормальный размер секций, например .text с размером "сырых" данных, равным нулю и ненулевым виртуальным размером

Также упакованные исполняемые файлы могут быть опознаны с помощью анализа их энтропии - показателя неопределенности. Сжатая или зашифрованная дата очень похожа на случайную данную, и поэтому имеет высокую энтропию. "Чистые" программы имеют низкую энтропию

2.2 Статический анализ упакованного ПО

Возьмем реальный образец вредоносного ПО, в котором для защиты от анализа применена техника упаковки, и проанализируем, как её можно детектировать, соотнося увиденную информацию с вышеизложенной теорией

Общая информация на рисунке 5 указывает на маленький размер файла и замаскированное время компиляции файла.

general		
stamp > compiler	0x00000000	Thu Jan 01 00:00:00 1970 (UTC)
size	0x14	20 bytes
file-header > location	0x00000084 - 0x00000098	0x00000084 - 0x00000098
signature	0x00004550	PE00
machine	0x014C	Intel-386
sections > count	0x0003	3
pointer-symbol-table	0x00B1E000	0x00000000 (expected)
number-of-symbols	0x00000000	0x00000000

Как видно на рисунке 6, файл имеет малое количество секций, нетипичное для PE-файла, и с названиями, которые указывают на упаковщик UPX. Причем один из разделов - с высокой энтропией.

property	value	value	value
section	section[0]	section[1]	section[2]
name	UPX0	UPX1	UPX2
section > unknown	n/a	n/a	n/a
entropy	n/a	8.000	1.470
file > ratio (99.98%)	n/a	99.97 %	0.02 %
raw-address (begin)	0x00000200	0x00000200	0x00323400
raw-address (end)	0x00000200	0x00323400	0x00323600
raw-size (3290112 bytes)	0x00000000 (0 bytes)	0x00323200 (3289600 bytes)	0x00000200 (512 bytes)
virtual-address (begin)	0x00001000	0x00867000	0x00B8B000
virtual-address (end)	0x00867000	0x00B8B000	0x00B8C000
virtual-size (12103680 bytes)	0x00866000 (8806400 bytes)	0x00001000 (4096 bytes)	

Исходя из информации об импортируемых функциях на рисунке 7, можно сказать о том, что их крайне мало, и все они отсылают к рантайм-распаковке:

imports (4)	flag (1)	type	ordinal	first-thunk (IAT)	first-thunk-original (INT)	library
LoadLibraryA	-	implicit	-	0x00B8B068	0x00000000	KERNEL32.DLL
ExitProcess	-	implicit	-	0x00B8B04A	0x00000000	KERNEL32.DLL
GetProcAddress	-	implicit	-	0x00B8B058	0x00000000	KERNEL32.DLL
VirtualProtect	x	implicit	-	0x00B8B076	0x00000000	KERNEL32.DLL

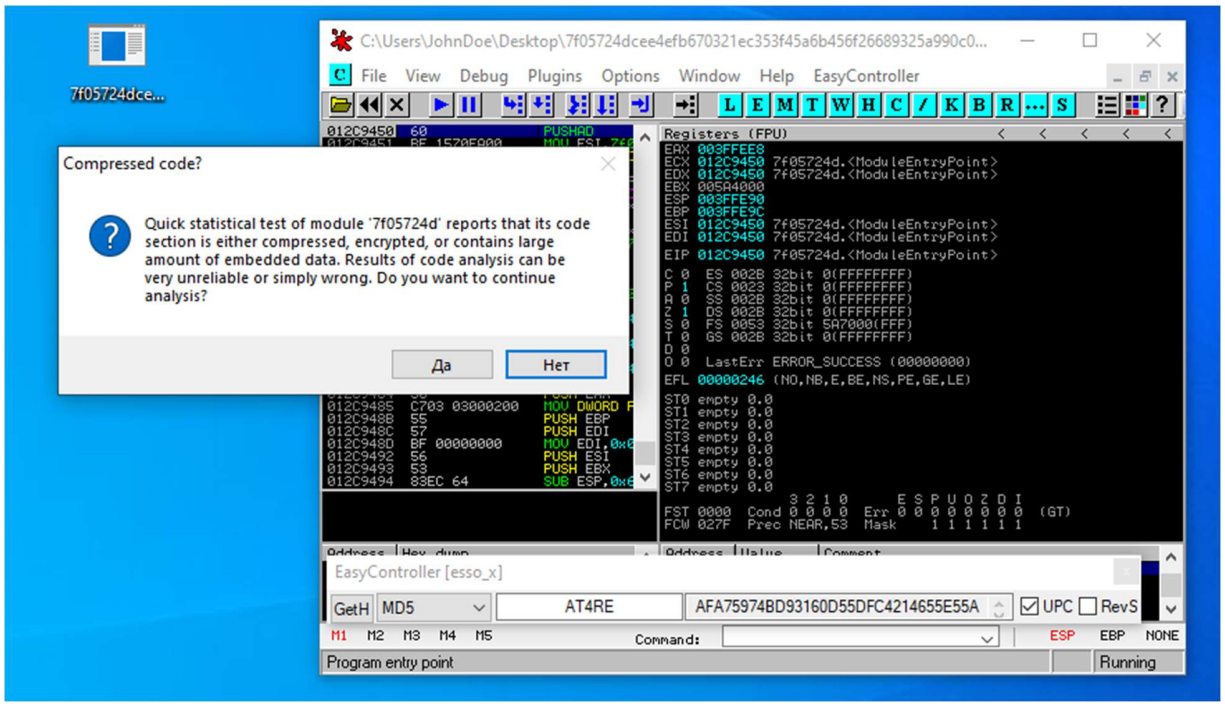
Таким образом, можно заключить, что образец, который анализируется, упакован. Попробуем пойти дальше, и провести статический и динамический анализ. Действительно, при попытке анализа через дизассемблер IDA Pro имеется большое количество непонятных, "мусорных", "сырых" конструкций, инструкций и данных, как на рисунке 8:

```
IDA View-A    Hex View-1    Local Types    Imports
UPX1:00C67000
UPX1:00C67000 ; Segment type: Pure code
UPX1:00C67000 ; Segment permissions: Read/Write/Execute
UPX1:00C67000 UPX1            segment para public 'CODE' use32
UPX1:00C67000                 assume cs:UPX1
UPX1:00C67000                 ;org 0C67000h
UPX1:00C67000                 assume es:nothing, ss:nothing, ds:UPX0, fs:nothing, gs:nothing
UPX1:00C67000                 dd 921E953Dh, 0B87BA9C4h, 32243000h, 0B1E20000h, 242600h
UPX1:00C67014                 db 36h
UPX1:00C67015 byte_C67015     db 1Ah, 3, 0            ; DATA XREF: start+1↓o
UPX1:00C67018                 dd 67008145h, 3835E5D4h, 0F3EA7918h, 7B375D3Ch, 0D420863Dh
UPX1:00C6702C                 dd 0F200D193h, 0DC90D191h, 402A4551h, 0F2453116h, 3DF4CAE3h
UPX1:00C67040                 dd 0C34BAFC9h, 0C7FC07B4h, 288C3A0h, 2E8354D0h, 0C4D72EE5h
UPX1:00C67054                 dd 944343E5h, 11C0E26Eh, 7A5A4373h, 8385C7E6h, 2C58C5EBh
UPX1:00C67068                 dd 3D69741Bh, 0DEEFAEDCh, 0C1409A71h, 6F5F8592h, 1E5E18D7h
UPX1:00C6707C                 dd 50504ACh, 49BF14D5h, 8F36FA3Dh, 119E2D96h, 0EB0833F1h
UPX1:00C67090                 dd 6EF808C9h, 7C014AEFh, 7C0F2EADh, 2A411AA1h, 4FDA1822h
UPX1:00C670A4                 dd 0AC2CCB41h, 0BDAE9EB2h, 208B8A9Eh, 1364C5FCh, 9A448583h
UPX1:00C670B8                 dd 0CC588713h, 514DF41Ah, 50B9682Ah, 6F34FB16h, 0A223FFC5h
UPX1:00C670CC                 dd 0A97A8C3Ah, 1AC9A709h, 3EFBB75Bh, 0FED242F3h, 7EDD5EEAh
UPX1:00C670E0                 dd 5AEEE07Eh, 77F31D22h, 370CCBECh, 99B03CE4h, 648F24EAh
UPX1:00C670F4                 dd 0B441828Fh, 9C78E5E6h, 0F297829Eh, 55082641h, 6CF8F8ADh
UPX1:00C67108                 dd 0EFB68181h, 0E7A9F639h, 3AA4EDF5h, 0DF607E0Fh, 87492FC7h
UPX1:00C6711C                 dd 242CEE85h, 0CB05CA79h, 5DDCAD58h, 0F438F394h, 3BA7E133h
UPX1:00C67130                 dd 7D419955h, 46D7D808h, 0B33A8E76h, 2A276078h, 0B4436EBBh
UPX1:00C67144                 dd 0CEE97B4Fh, 90C0AF03h, 14895C5Bh, 3961A10Ah, 0AF942028h
UPX1:00C67158                 dd 631569BBh, 1C1681ADh, 0C7FD56E3h, 89CC9644h, 0E84C5DF6h
UPX1:00C6716C                 dd 0EC8226B4h, 621A80CEh, 22026B12h, 0D2F0D4F7h, 77613E75h
UPX1:00C67180                 dd 34E64ACCh, 74371803h, 97E1C68Ah, 2465593h, 89F59Bh
UPX1:00C67194                 dd 57D1A0CAh, 0AC074CF4h, 42E005BCh, 5C227D6Ah, 0D33A599h
UPX1:00C671A8                 dd 1ED4C389h, 0ED7EDA9Dh, 70E46638h, 0EB5E2AE5h, 60C04A41h
UPX1:00C671BC                 dd 0B90D029Dh, 0EA002919h, 75ACB314h, 0CFF65921h, 753F0777h
UPX1:00C671D0                 dd 5A3A677Ah, 0C80159E1h, 0E6502513h, 93E4AF48h, 7BDFEB87h
UPX1:00C671E4                 dd 207E111Bh, 0CA51E8C2h, 0FA17FB8Bh, 0FD1623F1h, 5B8A257Ah
UPX1:00C671F8                 dd 0FB3D2B4Ch, 5F6B5B85h, 94181B11h, 0F7D5356Ah, 5D1EF3F6h
UPX1:00C6720C                 dd 5DCDA01h, 632D2275h, 423D8D82h, 0C6AA49C8h, 2D8BF41Ah
UPX1:00C67220                 dd 0ABEAB4FAh, 0BA7C2CB8h, 7C9FC5C0h, 2BC83720h, 6264AD03h
UPX1:00C67234                 dd 0A35A4E3Ch, 0E853BBEEh, 0D7F64113h, 153060DCh, 1231CCE5h
UPX1:00C67248                 dd 0D5D05E14h, 0F247D481h, 5F9069ECh, 0F303C885h, 0DF13A9AFh
```

А экспортируемая секция start на рисунке 9 является кодом заглушки:

```
UPX1:00F89450                 pusha
UPX1:00F89451                 mov     esi, offset byte_C67015
UPX1:00F89456                 lea     edi, [esi-866015h]
UPX1:00F8945C                 push    edi
UPX1:00F8945D                 mov     ebp, esp
UPX1:00F8945F                 lea     ebx, [esp+24h+var_3EA4]
UPX1:00F89466                 xor     eax, eax
UPX1:00F89468
UPX1:00F89468 loc_F89468:                             ; CODE XREF: start+1B↓j
UPX1:00F89468                 push    eax
UPX1:00F89469                 cmp     esp, ebx
UPX1:00F8946B                 jnz     short loc_F89468
UPX1:00F8946D                 inc     esi
UPX1:00F8946E                 inc     esi
UPX1:00F8946F                 push    ebx
UPX1:00F89470                 push    offset byte_B87BA9
UPX1:00F89475                 push    edi
UPX1:00F89476                 add     ebx, 4
UPX1:00F89479                 push    ebx
UPX1:00F8947A                 push    32242Eh
UPX1:00F8947F                 push    esi
UPX1:00F89480                 add     ebx, 4
UPX1:00F89483                 push    ebx
UPX1:00F89484                 push    eax
UPX1:00F89485                 mov     dword ptr [ebx], 20003h
UPX1:00F8948B                 push    ebp
UPX1:00F8948C                 push    edi
UPX1:00F8948D                 mov     edi, 0
UPX1:00F89492                 push    esi
0032268D 00F8948D: start+3D (Synchronized with Hex View-1)
```

При попытке запуска файла через отладчик OllyDbg на рисунке появляется соответственное предупреждение, как на рисунке 10:

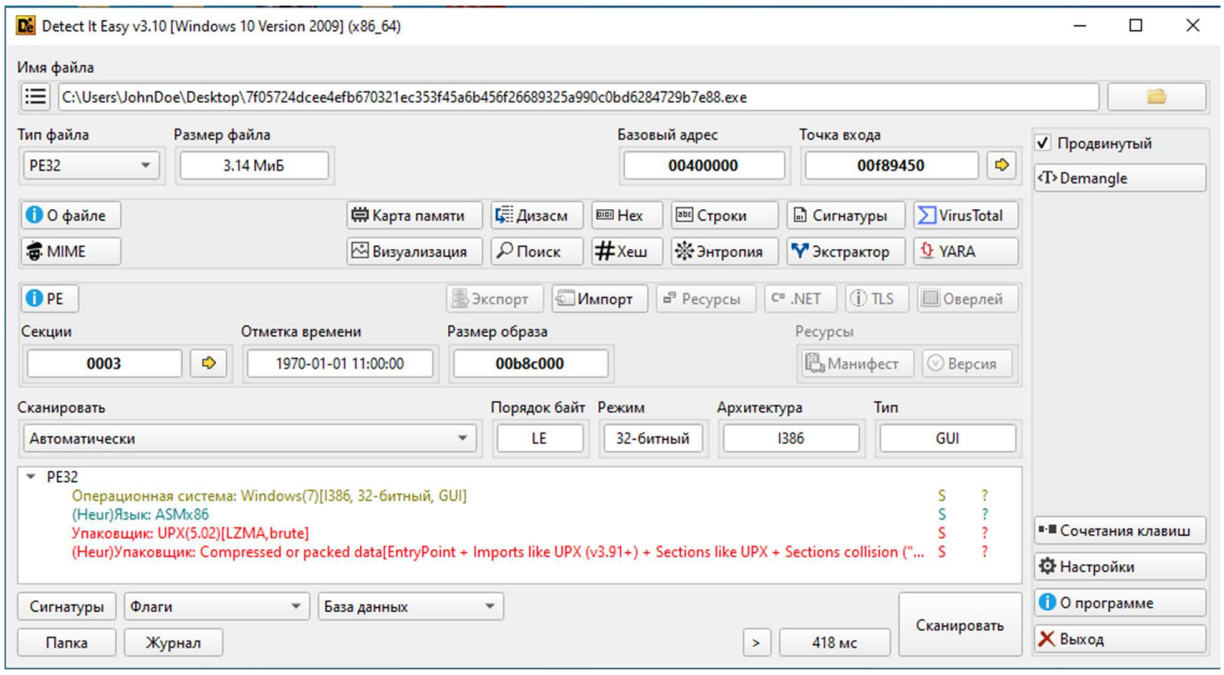


Таким образом, можно заключить, что как по отдельности, так и совокупности данные признаки позволяют сказать о том, что файл упакован с помощью упаковщика UPX.

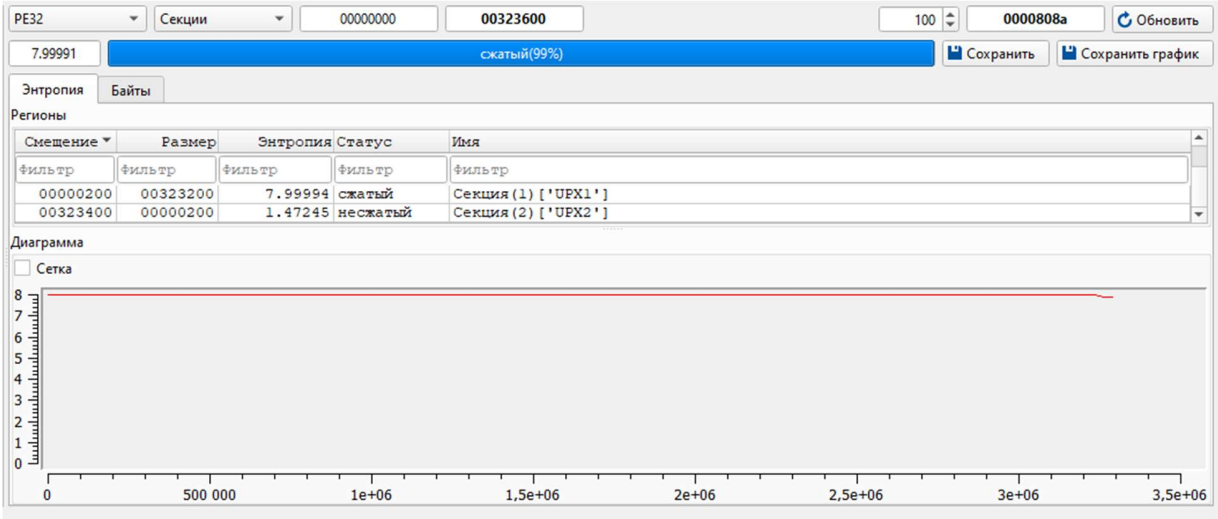
2.3 Автоматизированное детектирование упаковки

Так как статический анализ занимает большое количество времени и требует некоторых специализированных знаний для корректной интерпретации собранной информации, имеет смысл использовать автоматизированные инструменты статического анализа, которые могут выявлять упаковку. Рассмотрим один из таких инструментов - DetectItEasy, на рассмотрении такой же образец ВПО, как в предыдущем пункте.

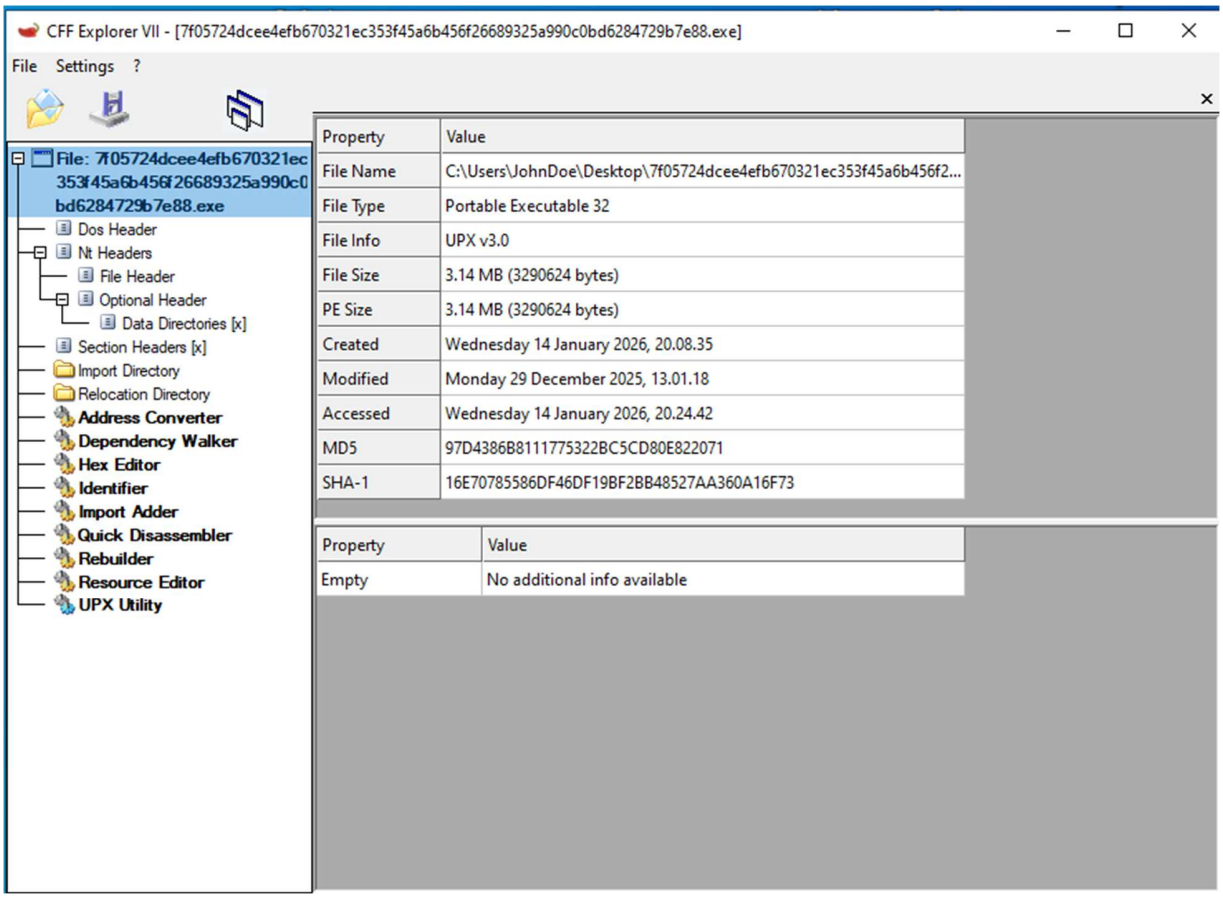
На рисунке 11 представлен результат автоматического сканирования инструментом:



То есть сразу видна информация о том, какой упаковщик с каким конкретно алгоритмом сжатия и опцией упаковки применялся, какие эвристические методы на это указывают - и это соотносится с признаками, указанными в теоретической части и начале данного раздела. Можно детальнее изучить информацию, например, как на рисунке 12, об энтропии по секциям, с их статусами, значениями и графиком:



Другой инструмент - CFF Explorer, также показывает аналогичные выводы, что можно увидеть на рисунке 13:



Если изучать дальнейшую информацию, то она будет полностью или практически полностью соответствовать полученным данным от DetectItEasy

и собранным с помощью ручного статического анализа. Кроме того, данный инструмент предоставляет встроенную утилиту для распаковки программ, упакованных упаковщиком UPX.

2.4 О распаковке и восстановлении оригинальной программы

Существует три общих подхода к распаковке упакованного исполняемого файла - автоматическая статическая, автоматическая динамическая, и ручная динамическая распаковки. Техники автоматической упаковки наиболее быстрые и легкие, чем техники распаковки вручную, но не всегда работают. Если упаковщик известен - для него существует автоматический распаковщик. Однако иногда разработчики вредоносов пишут пользовательские упаковщики, для которых могут не создавать публичные распаковщики.

Стоит упомянуть, что в контексте анализа вредоносного ПО, целью служит изучение поведения вируса, а не его воссоздание. Поэтому при распаковке ВПО получается новый двоичный файл, который не идентичен исходному, но который работает точно так же.

Исходя из простоты подхода, методы автоматической распаковки не рассматриваются. Куда больший практический интерес представляет подход, при котором распаковка происходит вручную.

Существует два распространённых подхода для осуществления распаковки вручную.

Первый подход заключается в изучении алгоритма упаковки и написания программы для распаковки в обратном порядке. Запуская исходный алгоритм в обратном направлении, каждый из шагов упаковки будет отменен.

Средства автоматической распаковки используют такой же подход, но он оказывается неэффективным, когда когда упаковщик пользовательский или слишком специфичный для того, чтобы реверсить алгоритм. Так что даже с автоматизацией этого процесса он может занять больше количество времени.

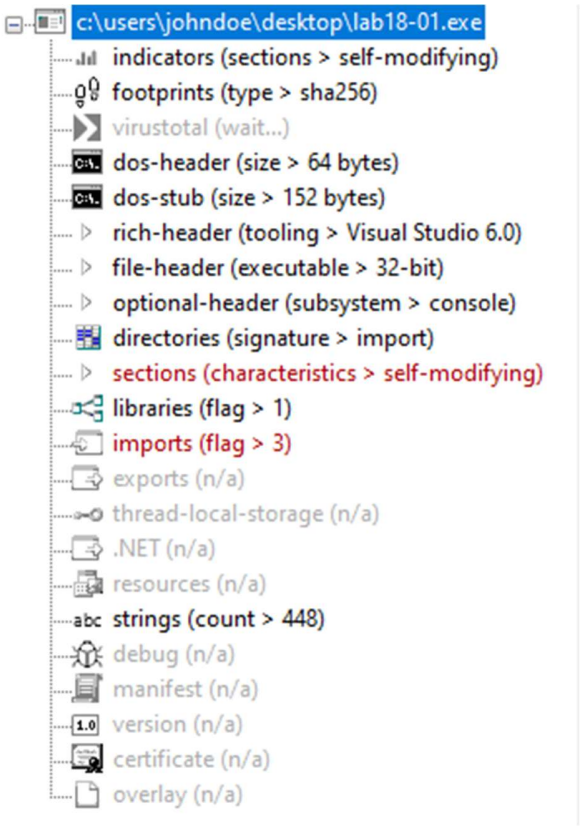
Второй подход заключается в том, что запустить программу и остановить её на этапе после распаковки распаковочной заглушкой. Тогда необходимо сдампить процесс из памяти, и вручную восстановить PE-заголовки и таблицу импорта при необходимости.

Так как каждый из упаковщиков на практике требует модификации общего подхода, рассмотрим варианты ручной распаковки, не требующих особой сложности вроде ручной реконструкции таблиц импорта и корректировании заголовков исполняемого файла. Рассмотрим на примере вредоносных образцов, упакованных с помощью UPX, но не содержащих других методов анти-анализа и анти-отладки техники ручной упаковки

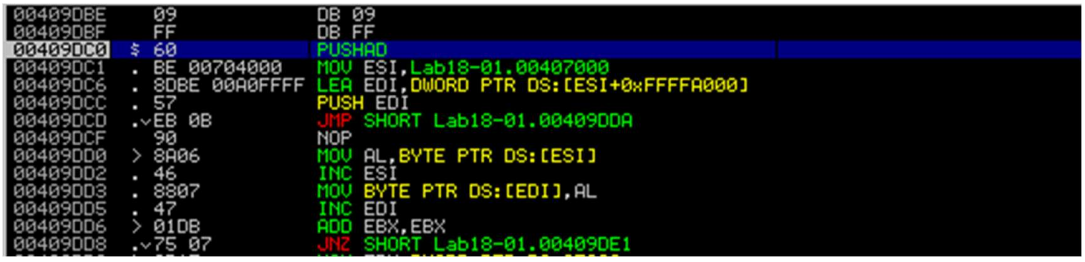
2.5 Tail Jump Catching

На самом деле, каждый раз, когда речь идёт об упакованном ВПО, при ручной распаковке необходимо просто остановиться в точке, где упакованный код уже распакован и находится в памяти, но не выполнен. Да, необходимо найти OEP и остановиться на ней. Как было описано ранее, управление на OEP передается с помощью паттерна tail jump. В данном случае, техника\способ Tail Jump Catching заключается в особенностях работы упаковщиков со стеком программы, и поэтому OEP можно поймать несколькими способами. Говоря о Tail Jump Catching, надо сказать, что хотя любой из подходов несёт в себе такую же цель, в данном случае нет необходимости следовать за вызовами каких-либо конкретных функций.

Возьмем явно упакованный файл. Так как сравнение упакованного и распакованного образцов в деталях будет весьма объемным, обзорно представим некоторые эвристические характеристики файла на рисунке 14:



Наша задача - найти OEP и остановить в этой точке исполнение файла. При запуске через отладчик OllyDbg мы останавливаемся на Entry Point распаковочной заглушки - на это указывается специфическая инструкция pushad, как на рисунке 15.



Фактически, при выполнении инструкции lea загрузится из памяти адрес OEP или конца заглушки, но если мы перейдем по нему сейчас, то увидим запакованные инструкции, как на рисунке 16:

```
00401000   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
00401002   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
00401004   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
00401006   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
00401008   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
0040100A   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
0040100C   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
0040100E   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
00401010   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
00401012   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
00401014   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
00401016   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
00401018   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
0040101A   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
0040101C   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
0040101E   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
00401020   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
00401022   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
00401024   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
00401026   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
00401028   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
0040102A   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
0040102C   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
0040102E   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
00401030   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
00401032   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
00401034   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
00401036   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
00401038   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
0040103A   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
0040103C   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
0040103E   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
00401040   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
00401042   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
00401044   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
00401046   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
00401048   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
0040104A   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
0040104C   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
0040104E   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
00401050   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
00401052   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
00401054   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
00401056   0000   add byte ptr ds:[eax],al   eax:&"яяяяГЪ"w"
```

Так как адрес OEP находится в стеке, а память в данном случае RWX - то есть заглушка запишет в память и из этой памяти потом будет выполняться программа, то отследить оригинальную точку входа можно с помощью hardware breakpoint по доступу к участку памяти, куда загружается адрес. Нас интересует адрес, записанный в регистр ESP на рисунке 17.

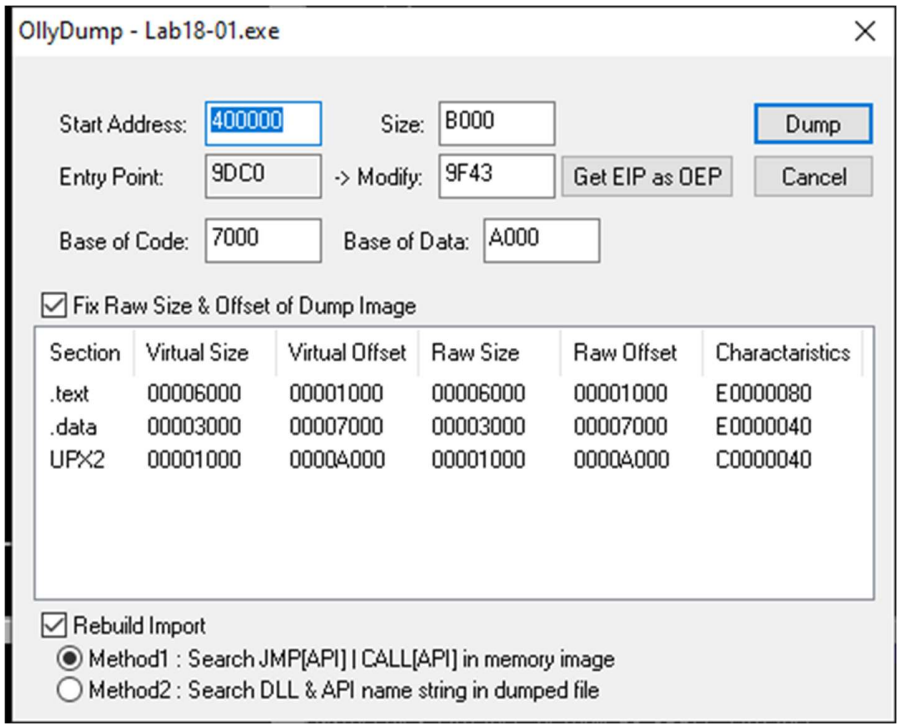
```
00409DBE    09          DB 09
00409DBF    FF          DB FF
00409DC0  $ 60          PUSHAD
00409DC1    BE 00704000 MOV ESI,Lab18-01.00407000
00409DC6  . 8DBE 00A0FFFF LEA EDI,DWORD PTR DS:[ESI+0xFFFFA000]
00409DCC  . 57          PUSH EDI
00409DCD  .vEB 0B       JMP SHORT Lab18-01.00409DDA
00409DCF  . 90          NOP
00409DD0  > 8A06        MOV AL,BYTE PTR DS:[ESI]
00409DD2  . 46          INC ESI
00409DD3  . 8807        MOV BYTE PTR DS:[EDI],AL

Registers (FPU)
EAX 0019FFCC
ECX 00409DC0 Lab18-01.<ModuleEntryPoint>
EDX 00409DC0 Lab18-01.<ModuleEntryPoint>
EBX 002EE000
ESP 0019FF74
EBP 0019FF80
ESI 00409DC0 Lab18-01.<ModuleEntryPoint>
EDI 00409DC0 Lab18-01.<ModuleEntryPoint>
EIP 00409DC0 Lab18-01.<ModuleEntryPoint>
```

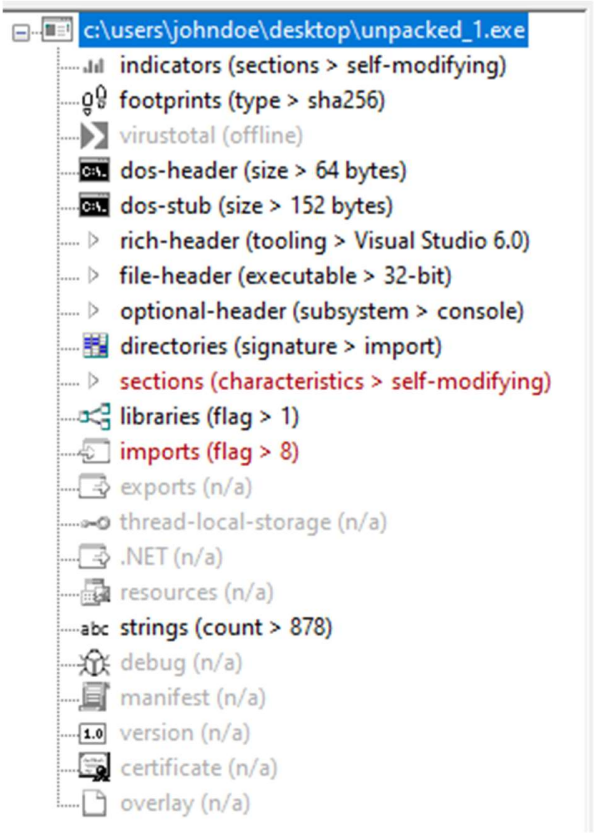
Перейдя к этому адресу в дампе памяти, устанавливается hardware breakpoint on access as DWORD. После запуска программы выполнение остановится прямо перед OEP, как на рисунке 18:

```
00409F31  . 57            PUSH EDI
00409F32  . FFD5          CALL EBP
00409F34  . 58            POP EAX
00409F35  . 61            POPAD
00409F36  . 8D4424 80     LEA EAX,DWORD PTR SS:[ESP-0x80]
00409F3A  > 6A 00         PUSH 0x0
00409F3C  . 39C4          CMP ESP,EAX
00409F3E  .^75 FA         JNZ SHORT Lab18-01.00409F3A
00409F40  . 83EC 80       SUB ESP,-0x80
00409F43  .-E9 0776FFFF   JMP Lab18-01.0040154F
```

Это - в чистом виде паттерн tail jump. Поставив точку останова на последнюю в блоке инструкцию jmp, и запустив программу, пока она не остановится в ней, мы добьемся нашей цели - программа уже распакована и находится в памяти, но не выполнена. Теперь с помощью плагина OllyDump сдампим процесс из памяти. На рисунке 19 показана работа плагина - он дампит процесс, меняя Entry Point.



Сохранив распакованный файл, посмотрим те же характеристики на рисунке 20, которые брали у запакованного образца, и увидим, что файл распакован.



2.6 VirtualProtect Following

Исходя из специфики работы упаковщиков, в них должен быть встроен обработчик исключений, полагающийся на работу Windows API любого уровня - пользовательского или ядра. В большинстве упаковщиков используется VirtualProtect. Говоря простыми словами, если упаковка не удалась - то процесс убивает сам себя, обрабатывая исключение или ошибку при распаковке. Отслеживание процесса вызовов этой функции и анализ возврата после возврата из неё вместе с внимательностью позволит найти tail jump паттерн и OEP как следствие. Возьмем тот же упакованный образец, что и в предыдущем пункте, и для разнообразия, проделаем процесс в отладчике x32dbg.

В первую очередь, необходимо остановиться на вызове функции VirtualProtect. В большинстве отладчиков есть окно ввода команд, и так как адрес вызова нам неизвестен, а поиск вручную займет слишком много времени, то команда bp VirtualProtect остановит выполнение на её вызове. В результате, как на рисунке 21, видим вызов и работу с этой функцией:

```
757F04C0   8BFF                 mov edi,edi                          VirtualProtect
757F04C2   55                   push ebp                             ebp:VirtualProtect
757F04C3   8BEC                 mov ebp,esp                          ebp:VirtualProtect
757F04C5   5D                   pop ebp                              ebp:VirtualProtect
757F04C6 - FF25 90138575        jmp dword ptr ds:[<VirtualProtect>]  JMP.&VirtualProtect
```

Дальше необходимо использовать "выполнение до возврата", и отладчик остановится на инструкции, возвращающей поток управление в другую функцию. Каждый раз после возврата из Virtual Protect необходимо внимательно анализировать, куда программа возвращается - таких вызовов и переходов может быть несколько, и можно пропустить tail jump

Как показано на рисунке 22, первый возврат не отсылает нас к tail jump и OEP, а к следующей секции или этапу распаковки - с новым вызовом VirtualProtect:

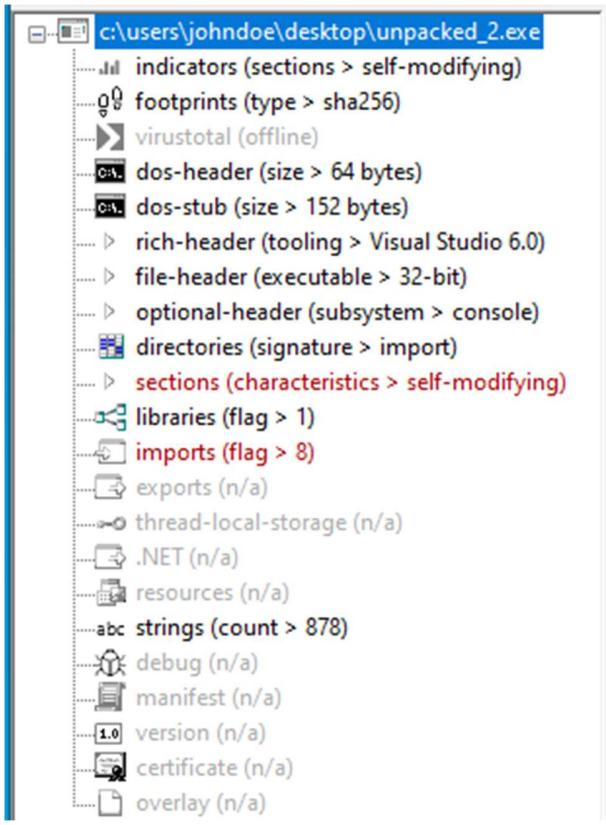
```
00409F1F   8D87 F7010000        lea eax,dword ptr ds:[edi+1F7]       edi+1F7:"a.data"
00409F25   8020 7F              and byte ptr ds:[eax],7F
00409F28   8060 28 7F           and byte ptr ds:[eax+28],7F
00409F2C   58                   pop eax
00409F2D   50                   push eax
00409F2E   54                   push esp
00409F2F   50                   push eax
00409F30   53                   push ebx
00409F31   57                   push edi                             edi:"MZЂ"
00409F32   FFD5                 call ebp                             ebp:VirtualProtect
00409F34   58                   pop eax
00409F35   61                   popad
00409F36   8D4424 80            lea eax,dword ptr ss:[esp-80]
00409F3A   6A 00                push 0
00409F3C   39C4                 cmp esp,eax
00409F3E ^ 75 FA                jne lab18-01.409F3A
00409F40   83EC 80              sub esp,FFFFFF80
00409F43 - E9 0776FFFF          jmp lab18-01.40154F
```

Снова выполняя программу, она остановится после вызова VirtualProtect и процесс повторяется.

После второго вызова на рисунке 23 видно, что мы находимся как раз в самом начале tail jump:

```
00409F2D   50                 push eax
00409F2E   54                 push esp
00409F2F   50                 push eax
00409F30   53                 push ebx
00409F31   57                 push edi                    edi:"MZЋ"
00409F32   FFD5               call ebp                    ebp:VirtualProtect
00409F34   58                 pop eax
00409F35   61                 popad
00409F36   8D4424 80          lea eax,dword ptr ss:[esp-80]
00409F3A   6A 00              push 0
00409F3C   39C4               cmp esp,eax
00409F3E   75 FA              jne lab18-01.409F3A
00409F40   83EC 80            sub esp,FFFFFF80
00409F43   E9 0776FFFF        jmp lab18-01.40154F
```

Теперь ставим точку останова на последней инструкции jump - передаче управления OEP, и дампим процесс. На рисунке 24 представлены характеристики сдампленного файла и тоже можно сделать вывод об успешной распаковке:



ЗАКЛЮЧЕНИЕ

В ходе выполнения данной курсовой работы:

А) была изучена архитектура исполняемых файлов формата Portable Executable ОС Windows, включая ключевые структуры заголовков и секций, критически важные для понимания методов их модификации;

Б) были исследованы принципы, методы и цели упаковки исполняемых файлов, проведена классификация упаковщиков, детально разобрана анатомия упаковщика;

В) были изучены и освоены на практике основные методы статического и динамического анализа для детектирования упаковки, а также специализированный инструмент для анализа;

Г) получен практический опыт ручной распаковки вредоносных образцов с применением базовых методик, включая дамп памяти процесса и проверку корректности восстановления исполняемого файла.

Поставленные задачи и цели достигнуты, а предложенное задание выполнено в полном объёме.

Результаты и ход выполнения данной курсовой работы демонстрируют степень усвоения студентом сложного теоретического материала, выходящего за рамки базового курса предмета, а также способность к его самостоятельному практическому применению. Работа показывает владение профессиональным инструментарием и методиками, актуальными для сферы информационной безопасности и анализа вредоносного кода.

Данная работа может служить основой для учебно-методического пособия по анализу вредоносного ПО или реверс-инженерии или может быть

расширена до полноценного научно-исследовательского проекта по изучению методов противодействия современным техникам обфускации и анти-анализа.

Список использованных источников

1. Сикорски М., Хониг А. Practical Malware Analysis: The Hands-On Guide to Dissecting Malicious Software. – Сан-Франциско: No Starch Press, 2012. – 800 с.

2. Дэмс Т., Селарадж А. Practical Reverse Engineering: x86, x64, ARM, Windows Kernel, Reversing Tools, and Obfuscation. – Индианаполис: Wiley, 2014. – 384 с.

3. Эйлам Э. Reversing: Secrets of Reverse Engineering. – Индианаполис: Wiley, 2005. – 595 с.

4. Microsoft. Microsoft Portable Executable and Common Object File Format Specification. Редакция 8.3. – 2022. – 140 с.

5. Eagle C. The IDA Pro Book: The Unofficial Guide to the World's Most Popular Disassembler. 2nd ed. – San Francisco: No Starch Press, 2011. – 640 с.

6. Yuschuk O. OllyDbg Documentation. – 2020.

7. National Institute of Standards and Technology Special Publication 800-83 Revision 1. Guide to Malware Incident Prevention and Handling for Desktops and Laptops // NIST SP 800-83rev1. – 2013. – 47 с.

8. x64dbg. Official Documentation. – Режим доступа: https://x64dbg.readthedocs.io/.